Duplicate finder/deleter/collector

Another good project to do is make your own system utilities for simple but useful tasks such as duplicate deletion. Redundancy is useful for data security but doesn't make much sense on the same physical drive. Besides, who doesn't want more storage space?! Why pay for file cleaning software if you can build it yourself?

You have to be really careful for this one though, one misplaced extra indent could lead to a bad accident and loss of data. We would recommend it for intermediate python programmers, not if this FastTrack is the first time you're diving into python. In that case, you can skip ahead to the next project, especially if you still want the challenge.

Another concept involved in this project is hashing. Normally used in cryptography because of the ease of calculating the function one way (and difficulty to reverse said calculation), we can use the unlikeliness of output collisions to use the hash of data as its fingerprint, instead of having to compare each location of memory, which would be highly inefficient.

For hashing, the md5 module would be the popular choice, for reasons you will learn as you build this project.

- **Step 1:** Get to know the basic file handling functionalities of the os module, and the path finding capabilities of glob. Practice creating a dummy file, deleting it, looping through the creation of a file ten times with ordered names. Make a temp directory and move the ten files there. Delete every second one. Make five subdirectories in the temp folder and move one file to each. Rename the files from a random list/iterator. Put some text in each - the system time at that instant, measured randomly between 20-40 times (separated by 5 ms between the consecutive measurements). Rename the subdirectories according to the file size inside it (look up the stat module for this). Delete the temp directory.
- **Step 2:** If you did all that, learning how to hash will take you ten minutes. Since identical files may not have identical filenames, structure your program as follows: Walk the chosen directory recursively and store locations and sizes of all the files (sorted by size). Among all files have the same sizes (sub-loop), hash the first megabyte or so of the files in a group and compare. Compare hashes of the complete file to confirm collisions and store these separately to output at the end.
- **Step 3:** Add options to delete every nth occurrence of each duplicate, etc, or a move to a temp folder for manual checking (in which case, store the original locations of the files so you can put back whatever is left